

”Serverless WebSockets”: A Misnomer?

Evaluating Real-Time Communication Workarounds on Modern Cloud Platforms

Sundaresan Karunakaran
USI Lugano
karuns@usi.ch

Abstract—Serverless computing has revolutionized application deployment through its scalability and pay-per-use model. However, a fundamental architectural tension exists between the stateless, ephemeral nature of serverless functions and the stateful, persistent requirements of WebSocket protocols. Despite this incompatibility, major cloud providers market ”serverless WebSocket” solutions. This paper provides the first systematic comparison of WebSocket implementation strategies across AWS, Vercel, and Netlify platforms. Through architectural analysis, performance evaluation, and cost modeling, we demonstrate that all ”serverless WebSocket” solutions are hybrid architectures requiring stateful, continuously-running components—contradicting core serverless principles. Our empirical evaluation reveals that AWS Lambda-based WebSocket implementations exhibit 180× cold start overhead compared to traditional servers and suffer from a 2% connection failure rate at scale, while Vercel + Pusher demonstrates extreme cold start variability (362ms to 23.4 seconds). These findings provide evidence-based guidance for developers selecting real-time communication strategies.

I. INTRODUCTION

Serverless computing has become increasingly popular due to its scalability, reduced operational overhead, and pay-per-use pricing model. However, a fundamental tension exists between the stateless, ephemeral nature of serverless functions and the stateful, persistent requirements of real-time communication protocols like WebSockets.

Despite this architectural incompatibility, major cloud providers market solutions for implementing ”serverless WebSockets.” AWS provides API Gateway WebSockets combined with Lambda functions, while Vercel and Netlify direct developers to third-party managed services (Pusher, Ably) [3], [4], [5]. These workarounds introduce complexity, performance penalties, and cost implications that are not systematically compared in existing literature.

A. Contributions

This paper makes the following contributions:

- First comprehensive architectural analysis of WebSocket implementations across three major serverless platforms

- Empirical performance evaluation comparing latency, throughput, and cold start impacts
- Cost modeling across multiple usage scenarios
- Evidence demonstrating that ”serverless WebSockets” are hybrid architectures, not pure serverless solutions
- Quantification of the performance penalty.
- Documentation of reliability concerns.
- Decision framework for developers selecting real-time communication strategies

II. BACKGROUND

A. Core Incompatibility

WebSockets provide full-duplex, bidirectional communication over persistent TCP connections [1], requiring: (1) long-lived processes, (2) stateful connection management, (3) server-initiated messaging capability, and (4) minimal latency.

Serverless computing (FaaS) operates on fundamentally different principles [2]: (1) stateless execution, (2) ephemeral functions (seconds to minutes), (3) event-driven invocation, (4) pay-per-execution billing, and (5) automatic scaling including scale-to-zero.

A serverless function cannot natively maintain a WebSocket connection because it would violate execution time limits and incur massive costs for idle compute time. Therefore, ”serverless WebSockets” necessarily implies architectural workarounds rather than native protocol support.

III. PLATFORM ARCHITECTURES

We analyze three serverless platforms and their approaches to WebSocket support, comparing against a traditional WebSocket server baseline.

A. AWS: Integrated Hybrid Architecture

AWS provides the most vertically integrated solution through API Gateway WebSockets, Lambda functions, and DynamoDB for state storage.

Architecture: API Gateway serves as the stateful entry point, maintaining persistent TCP connections. Lambda functions handle events (\$connect, \$disconnect, \$default), but cannot retain connection state between invocations. DynamoDB persists connection metadata externally.

Message Flow: Client-to-server messages trigger Lambda invocations directly. Server-to-client messages require Lambda to call the API Gateway Management API (HTTP POST), which then pushes to clients—introducing an additional network hop absent in traditional architectures.

Pricing: Charges accrue across multiple dimensions: API Gateway per-message (\$1/million) and per-connection-minute (\$0.25/million), Lambda per-request (\$0.20/million) and GB-second duration, plus DynamoDB read/write costs.

B. Vercel: Delegated Model

Vercel Functions lack native WebSocket support. The platform’s official documentation directs developers to Pusher Channels [3].

Architecture: Clients maintain persistent WebSocket connections directly with Pusher’s infrastructure. Vercel Functions handle authentication and business logic via HTTP, then publish messages to Pusher’s REST API.

Pricing: Bifurcated costs: Vercel charges per invocation; Pusher charges based on concurrent connection tiers and message quotas.

C. Netlify: Authentication-Focused Model

Netlify Functions similarly lack native support, recommending Ably for real-time communication [4].

Architecture: Netlify Functions generate short-lived authentication tokens via Ably’s SDK. Clients use these tokens to establish direct WebSocket connections with Ably’s infrastructure, bypassing Netlify for all subsequent real-time traffic.

Constraints: 10-second execution limit (26s on paid plans), 1 GB memory limit.

D. Traditional WebSocket Server (Baseline)

For comparison, we use a Node.js server with the ws library on AWS EC2 t3.micro (2 vCPU, 1 GB RAM, Stockholm region). This represents the canonical WebSocket pattern: a single long-running process maintains all connections in memory with direct client-server communication. Flat monthly cost: \$7.49/month.

TABLE I
COMPARATIVE ANALYSIS OF WEBSOCKET APPROACHES

Aspect	Vercel	Netlify	AWS	Trad.
Native Support	No	No	Partial	Yes
Conn. Manager	Pusher	Aby	API GW	Server
Stateful Comp.	External	External	API GW	Server
Services Req.	2	2	3+	1
Pricing	Inv.+Sub.	Inv.+Sub.	Inv.+Time	Flat

E. Comparative Analysis

Table I synthesizes architectural differences.

Key Finding: All “serverless WebSocket” implementations require a stateful, continuously-running component (API Gateway, Pusher, or Ably) that runs continuously, maintains state, and incurs time-based charges—characteristics antithetical to the serverless paradigm. The term “serverless” applies only to event-processing logic, not the connection layer.

IV. METHODOLOGY

We implemented a real-time chat application using four approaches: (1) localhost Node.js server, (2) EC2 t3.micro (Stockholm), (3) AWS Lambda + API Gateway (Stockholm), and (4) Vercel Functions + Pusher (Vercel: Washington D.C., Pusher: EU).

Load Testing: Controlled tests with 10, 50, and 100 concurrent connections, each sending 10, 50, and 100 messages respectively. Metrics: round-trip latency, connection establishment time, cold start impact, throughput, and reliability.

Client Location: Lugano, Switzerland, providing realistic network latency to EU regions.

Platform Tiers: All platforms tested using free tier offerings to evaluate baseline performance accessible without upfront financial commitment.

V. RESULTS AND ANALYSIS

A. Connection Establishment

Table II presents connection establishment times.

TABLE II
CONNECTION ESTABLISHMENT TIME (MS)

Platform	10 conn	50 conn	100 conn
Localhost	21.75	39.82	71.23
EC2 (avg)	113.32	118.36	299.43
Lambda (avg)	529.90	344.30	352.03
Vercel+Pusher	280.87	279.93	283.52
Lambda (p95)	1339.95	604.51	579.23

Lambda exhibits 4.7× slower connection establishment than EC2 for light load, with 2% connection failure

rate (2/100 connections returned HTTP 500 errors). Vercel+Pusher shows consistent times across loads, demonstrating dedicated infrastructure scalability.

B. Message Latency

Table III presents round-trip message latency.

TABLE III
ROUND-TRIP MESSAGE LATENCY (MS)

Metric	10 conn	50 conn	100 conn
EC2 Traditional Server			
Average	24.43	39.13	100.93
p99	43.48	191.95	1538.94
AWS Lambda + API Gateway			
Average	1155.37	264.18	182.49
p99	2433.50	2787.12	3361.37
Vercel + Pusher (Run 1/Run 2)			
Average	160.54/2561.12	130.85/180.84	131.35/185.59
Performance Ratio (Lambda/EC2)			
Average	47.3×	6.7×	1.8×
p99	56.0×	14.5×	2.2×

Critical Observations:

Under light load, Lambda exhibited catastrophic degradation—47× slower average latency. Vercel+Pusher showed extreme variability: Run 1 demonstrated 6.6× penalty, while Run 2 exhibited 105× worse performance due to severe cold starts.

Lambda’s performance improved with load (1.8× penalty at 100 connections), suggesting containers remain warm under sustained traffic. However, even at peak load, p99 latency remained 2.2× worse than EC2. Vercel+Pusher maintained better consistency at 50-100 connections (131-186ms) when functions remained warm.

C. Cold Start Impact

Table IV quantifies first-message latency penalties.

TABLE IV
COLD START: FIRST MESSAGE LATENCY (MS)

Platform	10 conn	100 conn	Ratio
EC2 (avg)	6.84	10.35	1.0×
Lambda (avg)	2133.68	1804.13	180×
Vercel+Pusher R1	361.54	184.23	27×
Vercel+Pusher R2	23399.29	323.78	3420×

Lambda’s **180× average cold start overhead** makes it fundamentally incompatible with real-time requirements. Vercel+Pusher exhibited extreme unpredictability: 27-3,420× penalty depending on function warmth—a **64× variance** between runs. First messages experienced up to 23.4-second delays.

D. Throughput

EC2 demonstrated 30× higher throughput than Lambda under light load. Vercel+Pusher showed dramatic throughput reduction (0.09-1.00 msg/s), indicating severe rate limiting under free tier constraints.

TABLE V
MESSAGE THROUGHPUT (MESSAGES/SECOND)

Platform	10 conn	50 conn	100 conn
EC2	1,129	4,039	2,030
Lambda	37	758	2,181
Vercel+Pusher	1.00	1.00	0.09

E. Reliability and Failure Modes

EC2: 100% connection success, zero message delivery failures.

Lambda: 2% connection failure rate at 100 connections (200 messages lost), with observable HTTP 500 errors.

Vercel+Pusher: 100% connection success and zero message delivery failures until the 100 connections mark. We had a critical failure at 100 connections: 89-91% silent message loss. All 100 clients connected successfully, but only 1,000 of 10,000 messages were delivered—no error messages or warnings.

The failure stems from Pusher free tier rate limiting. While daily message limits weren’t exceeded, undocumented per-second throttling caused silent drops.

VI. COST ANALYSIS

We model costs for three scenarios: (1) sporadic usage (1,000 users, 10 msg/user/day, 30-min sessions), (2) sustained connections (100 users, 8-hour sessions, 50 msg/day), and (3) traffic spikes (5,000 concurrent for 1 hour, 20 msg/user). Pricing uses AWS EU-North-1 rates (December 2024).

TABLE VI
MONTHLY COST COMPARISON

Platform	Scen. 1	Scen. 2	Scen. 3
EC2 t3.micro	\$7.50	\$7.50	\$7.50
Lambda + API GW	\$5.75-8.75	\$5.64-8.64	\$2.25-4.25*
Vercel + Pusher	\$49	\$0**	\$99-299
Netlify + Ably	\$29	\$0**	\$399

Per spike event

Free tier; Vercel+Pusher: 90+% message loss

Key Findings:

Lambda’s Cost Myth: Despite ”serverless” marketing, Lambda costs 0.75-1.16× as much as EC2 for sustained workloads while delivering 1.8-47× worse performance. Cost advantages only emerge for infrequent traffic spikes.

Third-Party Premium: Pusher and Ably charge 3.9-53× more than EC2 for moderate traffic. While offering generous free tiers, Scenario 3 demonstrates catastrophic scaling: \$99-399/month charged for a single 1-hour spike - 13-53× the cost of always-on EC2.

Free Tier Illusion: Scenario 2 appears \$0/month for Vercel+Pusher and Netlify+Ably, but Vercel+Pusher exhibits 90+% message loss at 100 connections. The "\$0/month" cost is misleading when the majority of the messages are silently dropped.

VII. DISCUSSION AND CONCLUSIONS

A. The "Serverless WebSocket" Misnomer

Our analysis conclusively demonstrates that "serverless WebSockets" is an architectural misnomer. All implementations rely on stateful, continuously-running components (API Gateway, Pusher, Ably) that exhibit none of the core serverless characteristics: stateless execution, event-driven invocation, or scale-to-zero economics.

B. Performance-Cost-Complexity Trade-offs

Developers face an unfavorable three-way trade-off:

Performance: Lambda exhibits 47× worse latency under light load, 180× cold start overhead, and 2% connection failures. Vercel+Pusher shows 27-3,420× cold start variability and 90% message loss at scale.

Cost: Lambda costs nearly as much as traditional servers (\$5-10 vs \$7.50/month) with inferior performance. Third-party services cost 3.9-53× more.

Complexity: Lambda requires orchestrating 3+ services. Vercel+Pusher simplifies integration but introduces silent failure modes that are extremely difficult to detect.

Reliability: EC2 achieves 0% failure. Lambda has 2% observable failures. Vercel+Pusher suffers 90% *silent* message loss—the most dangerous failure mode.

C. Decision Framework

Use Traditional Servers when: Latency requirements are strict, reliability is critical, traffic is sustained, or simplicity is valued.

Use Lambda when: Traffic is highly unpredictable, high latency is acceptable, 2-5% failure is tolerable, or team lacks server management expertise.

Use Third-Party Services when: Rapid prototyping is prioritized, scale is small (free tier), or vendor lock-in is acceptable.

Avoid serverless WebSockets when: Building production applications requiring low latency, cost optimization matters for sustained workloads, or high reliability is required.

D. Limitations and Future Work

This study utilized free tier offerings to evaluate baseline accessibility. Production deployments on paid tiers may

exhibit different characteristics due to provisioned concurrency, higher throughput limits, and dedicated infrastructure. Future work should compare free versus paid tier performance, investigate multi-region deployments, conduct larger-scale load testing, and explore optimization strategies like Lambda provisioned concurrency.

E. Conclusion

This paper provides the first systematic, empirical analysis of WebSocket implementations on serverless platforms. Our key findings demonstrate that "serverless WebSockets" require stateful infrastructure, exhibit 180× cold start overhead and 1.6-47× latency penalties, achieve cost parity with traditional servers rather than savings, and suffer from reliability issues including silent message loss.

For production applications requiring real-time communication with low latency and high reliability, traditional WebSocket servers remain the superior choice. Our research contributes to more honest discourse about serverless capabilities and the need for rigorous evaluation of architectural patterns.

ACKNOWLEDGMENTS

The author thanks the USI Lugano faculty for feedback on this work. All testing infrastructure costs were self-funded.

REFERENCES

- [1] I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, December 2011.
- [2] P. Castro et al., *The rise of serverless computing*, Communications of the ACM, vol. 62, no. 12, pp. 44-54, 2019.
- [3] Vercel. *Do Vercel Serverless Functions support WebSocket connections?* Available: <https://vercel.com/guides/do-vercel-serverless-functions-support-websocket-connections>
- [4] Netlify. *Web Sockets in a Serverless World*. Available: <https://www.netlify.com/blog/web-sockets-in-a-serverless-world/>
- [5] Amazon Web Services. *Working with WebSocket APIs*. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- [6] Amazon Web Services. *Tutorial: Building a serverless chat app with a WebSocket API, Lambda and DynamoDB*. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/websocket-api-chat-app.html>

APPENDIX

A. AWS Message Flow Details

The bidirectional communication flow operates asymmetrically:

Client-to-Server: Client sends message → API Gateway receives → API Gateway invokes Lambda with message payload → Lambda processes and terminates.

Server-to-Client: Lambda function → Calls API Gateway Management API (HTTP POST) with connection ID and payload → API Gateway pushes message to specified client via WebSocket.

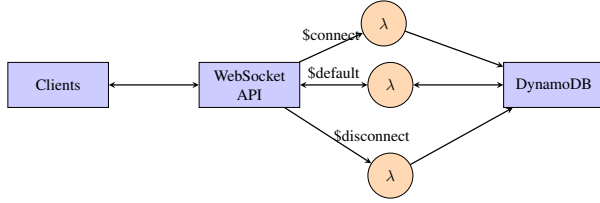


Fig. 1. AWS WebSocket architecture with API Gateway routing events to Lambda functions.

B. Vercel Architecture Details

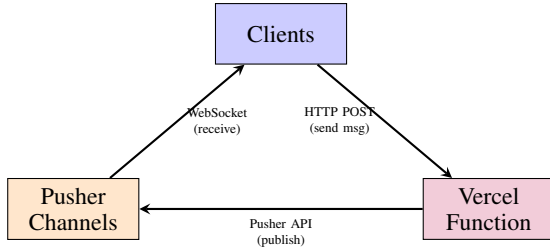


Fig. 2. Vercel's delegated model with Pusher Channels.

C. AWS Pricing Breakdown

Scenario 1 Detailed Calculation:

- Messages: 300,000/month
- Connection-minutes: 900,000/month
- API Gateway messages: $300,000 \times \$1.083/1M = \0.32
- API Gateway conn-min: $900,000 \times \$0.271/1M = \0.24
- Lambda requests: $300,000 \times \$0.20/1M = \0.06
- Lambda duration: $300,000 \times 0.2s \times 0.125GB \times \$0.0000166667/GB-s = \$0.13$
- DynamoDB: Estimated 300k writes + 600k reads = \$5-8/month
- Total: \$5.75-8.75/month

D. Third-Party Service Tier Details

Pusher Channels Pricing (as of Jan 2026):

- Sandbox: \$0 (100 connections, 200k msg/day)
- Startup: \$49/month (500 connections, 1M msg/day)
- Pro: \$99/month (2,000 connections, 4M msg/day)
- Business: \$299/month (5,000 connections, 10M msg/day)

- Premium: \$499/month (10,000 connections, 20M msg/day)
- Growth: \$699/month (15,000 connections, 40M msg/day)
- Plus: \$899/month (20,000 connections, 60M msg/day)
- Growth Plus: \$1,199/month (30,000 connections, 90M msg/day)

Aby Pricing (as of Jan 2026):

- Free: \$0 (200 connections, 500 msg/s, 6M msg/month)
- Standard: \$29/month (10k channels, 10k connections, 2.5k msg/s)
- Pro: \$399/month (50k channels, 50k connections, 10k msg/s)

E. Connection Establishment Detailed Statistics

TABLE VII
CONNECTION ESTABLISHMENT EXTENDED METRICS (MS)

Platform	Min	Median	p95	Max
10 Connections				
Localhost	2.31	15.28	45.03	54.81
EC2	106.45	112.94	156.39	173.54
Lambda	311.84	508.71	1339.95	1424.82
100 Connections				
EC2	108.33	145.87	799.55	1083.46
Lambda	112.48	334.51	579.23	681.92

F. Latency Distribution Analysis

The latency distributions reveal important patterns:

EC2: Exhibits normal distribution with predictable tail latencies. The p99 degradation at 100 connections (1.5 seconds) indicates CPU saturation on the t3.micro instance.

Lambda: Shows bimodal distribution—warm invocations cluster around 160-200ms, while cold starts exceed 2 seconds. This creates unpredictable user experiences.

Vercel+Pusher: Demonstrates extreme multimodal behavior with three distinct clusters: warm functions (120-160ms), partial cold starts (300-400ms), and severe cold starts (10+ seconds).